

DAY 1: Mobile OTP verification, FRONTEND

Angular 16 + Material, Midnight theme. **5 new files + 3 small edits**. Does not change any existing id or form field.

1. What the user sees

1. Register as a customer. If OTP is on, the page **opens** `/verify-phone` instead of the login page.
2. The page sends the code automatically, shows the email, and shows a 6-digit box (digits only; letters are removed as you type).
3. Wrong code: "Wrong code. 4 tries left." and the box clears. After 5 wrong codes: "Too many wrong codes. Request a new code."
4. **Resend** is greyed out with a countdown ("Resend in 27s").
5. Right code: "Mobile number verified", then the login page after 2.5 seconds.
6. If someone tries to log in before verifying, the backend answers 403 and the **login page forwards them to** `/verify-phone`.
7. If OTP is switched off (`OTP_REQUIRED=false`), `/verify-phone` asks the backend, sees `required: false` and **goes straight to the login page**, so the old flow still ends on login.

2. Honest status

Built and tested in a real browser (Puppeteer) against a pretend backend that follows the same API: **20 checks, 0 failures** (off and on, wrong code, countdown, resend, lock after 5, login redirect, no email, phone width, no JS errors). Not tested against the real Spring backend. If your SME's automated tests assert that registration navigates to `/login`, note that with OTP on, registration now shows the verification step on the same page; with OTP off it ends on `/login` exactly as before.

3. Edits to existing files

3a. `src/app/constant.ts`: add inside the `API` object (before its closing `}`):

```
// mobile OTP (Day 1)
OTP_STATUS: '/api/otp/status',
OTP_SEND: '/api/otp/send',
OTP_VERIFY: '/api/otp/verify',
```

3b. `src/app/app.module.ts`: add the import and put `VerificationModule` in imports **before** `AppRoutingModule` (the router takes the first matching route, and `AppRoutingModule` ends with a `**` catch-all):

```
import { VerificationModule } from '../modules/verification/verification.module';
// ...
imports: [BrowserModule, BrowserAnimationsModule, HttpClientModule, SharedModule, VerificationModule,
AppRoutingModule],
```

(If you already added `AiModule` , keep it in the list too: `SharedModule, AiModule, VerificationModule, AppRoutingModule` .)

3c. Registration page: the OTP step is shown INSIDE the registration page. Replace the three registration files with the ones below. After the person presses Register, the same card changes to "Step 2: Verify your mobile number" (code box, resend timer, and in dev mode a note with the code), then "Step 3: Done" with a Log in button. A small step list (1 Your details, 2 Verify mobile, 3 Done) is shown at the top whenever the server has OTP switched on. If OTP is switched off on the server, nothing changes: no step list, and Register ends on the login page as before.

components/registration/registration.component.ts (REPLACE the whole file)

```
import { AfterViewInit, Component, ElementRef, NgZone, OnDestroy, OnInit, ViewChild } from '@angular/core';
import { AbstractControl, FormBuilder, FormGroup, FormGroupDirective, NgForm, ValidationErrors, Validators }
  from '@angular/forms';
import { ErrorStateMatcher } from '@angular/material/core';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';
import { OtpService } from '../../services/otp.service';
import { CustomerService } from '../../services/customer.service';
import { GoogleAuthService } from '../../services/google-auth.service';

/** Username: no spaces anywhere. */
function noSpaces(control: AbstractControl): ValidationErrors | null {
  return /\s/.test(control.value || '') ? { spaces: true } : null;
}

/** Password: one upper case, one lower case, one number, one special character, and no spaces. */
export const PASSWORD_RULES = [
  { key: 'length', label: '8 to 72 characters', test: (v: string) => v.length >= 8 && v.length <= 72 },
  { key: 'upper', label: 'One upper case letter', test: (v: string) => /[A-Z]/.test(v) },
  { key: 'lower', label: 'One lower case letter', test: (v: string) => /[a-z]/.test(v) },
  { key: 'digit', label: 'One number', test: (v: string) => /[0-9]/.test(v) },
  { key: 'special', label: 'One special character (for example @ # $ % !)', test: (v: string) => /^[A-Za-z0-9\s]/.test(v) },
  { key: 'space', label: 'No spaces', test: (v: string) => v.length > 0 && !/\s/.test(v) }
];

function strongPassword(control: AbstractControl): ValidationErrors | null {
  const v: string = control.value || '';
  return v && PASSWORD_RULES.some(r => !r.test(v)) ? { weak: true } : null;
}

/** Email: name, one @, then a domain with a dot followed by at least 2 letters (a@b.com works, a@b does not). */
const EMAIL_PATTERN = /^[A-Za-z0-9._%+-]+@[A-Za-z0-9\-.]+\.(?=[A-Za-z0-9\-.]+)*\.[A-Za-z]{2,}$/;

/** Mobile: 10 digits, the first one is 6, 7, 8 or 9. */
const MOBILE_PATTERN = /^[6-9][0-9]{9}$/;

function passwordsMatch(group: AbstractControl): ValidationErrors | null {
  const p = group.get('password')?.value;
  const c = group.get('confirmPassword')?.value;
  return p && c && p !== c ? { mismatch: true } : null;
}

/** Shows the "Password do not match" error on the confirm field, even though the rule lives on the whole form. */
class ConfirmMatcher implements ErrorStateMatcher {
  isErrorState(control: AbstractControl | null, form: FormGroupDirective | NgForm | null): boolean {
    return !!control && (control.touched || control.dirty) && (control.invalid ||
      !!form?.form.hasError('mismatch'));
  }
}
```

```

@Component({
  selector: 'app-registration',
  templateUrl: './registration.component.html',
  styleUrls: ['./registration.component.css']
})
export class RegistrationComponent implements OnInit, OnDestroy, AfterViewInit {
  registerForm: FormGroup;
  errorMessage = '';
  loading = false;
  showPassword = false;
  passwordFocused = false;
  confirmMatcher = new ConfirmMatcher();

  // ---- mobile OTP, shown inside this page (step 2) ----
  /** null = not known yet, true = the server asks for OTP, false = OTP is switched off on the server. */
  otpRequired: boolean | null = null;
  step: 'form' | 'verify' | 'done' = 'form';
  email = '';
  maskedMobile = '';
  code = '';
  sending = false;
  verifying = false;
  otpError = '';
  otpInfo = '';
  devCode = '';
  secondsLeft = 0;
  private timer?: any;

  // ---- sign up with Google (OAuth): the account is created and stored in the database ----
  googleEnabled = false;
  googleLoading = false;
  googleError = '';
  /** True when the Google e-mail is already registered: the page then offers a link to log in. */
  alreadyRegistered = false;
  @ViewChild('googleButton') googleButton?: ElementRef<HTMLElement>;

  constructor(private fb: FormBuilder, private auth: AuthService, private router: Router, private otp:
OtpService,
    private google: GoogleAuthService, private customerService: CustomerService, private zone:
NgZone) {
    this.googleEnabled = this.google.enabled;
    this.registerForm = this.fb.group({
      username: ['', [Validators.required, Validators.minLength(3), noSpaces]],
      email: ['', [Validators.required, Validators.pattern(EMAIL_PATTERN)]],
      password: ['', [Validators.required, strongPassword]],
      confirmPassword: ['', [Validators.required]],
      mobileNumber: ['', [Validators.required, Validators.pattern(MOBILE_PATTERN)]],
      role: ['Customer'] // fixed: public sign-up always creates a customer
    }, { validators: passwordsMatch });
  }

  ngOnInit(): void {
    // only to show a short note and the step list; failing here changes nothing
    this.otp.status().subscribe({ next: s => (this.otpRequired = !!s.required), error: () =>
(this.otpRequired = null) });
  }

  ngAfterViewInit(): void {
    if (this.googleEnabled && this.googleButton) {
      this.google.renderButton(this.googleButton.nativeElement, (credential) =>
this.onGoogleCredential(credential), 'signup_with')
        .catch(() => { this.googleError = 'Google sign-up could not be loaded. Check your internet
connection.'; });
    }
  }

  ngOnDestroy(): void { clearInterval(this.timer); }

  /** Google calls this outside Angular. mode "register" makes the server store the account (name and e-
mail from Google). */
  private onGoogleCredential(credential: string): void {

```

```

this.zone.run(() => {
  if (this.googleLoading) { return; }
  this.googleLoading = true;
  this.googleError = '';
  this.alreadyRegistered = false;
  this.auth.loginWithGoogle(credential, 'register').subscribe({
    next: () => {
      // the account now exists and the user is logged in; a customer fills in his details once on the
next page
      this.googleLoading = false;
      this.customerService.viewCustomerByUserId().subscribe({
        next: () => this.router.navigate(['/dashboard']),
        error: () => this.router.navigate(['/customerdashboard'])
      });
    },
    error: (err) => {
      this.googleLoading = false;
      if (err.status === 409) {
        this.alreadyRegistered = true;
        this.googleError = 'This Google e-mail is already registered. Please log in instead.';
      } else if (err.status === 401) {
        this.googleError = 'Google sign-up failed. Please try again.';
      } else if (err.status === 400) {
        this.googleError = 'Sign up with Google is not set up on the server yet.';
      } else {
        this.googleError = 'Unable to reach the server. Please try again later.';
      }
    }
  });
});
});
}

get f() { return this.registerForm.controls; }

/** Live checklist under the password field. */
get passwordChecks() {
  const v: string = this.f['password'].value || '';
  return PASSWORD_RULES.map(r => ({ label: r.label, ok: r.test(v) }));
}

onSubmit(): void {
  if (this.registerForm.invalid || this.loading) { return; }
  this.loading = true;
  this.errorMessage = '';
  const { confirmPassword, ...user } = this.registerForm.value;
  this.auth.register(user).subscribe({
    next: () => this.afterRegister(user.email, user.mobileNumber),
    error: (err) => {
      this.loading = false;
      if (err.status === 409) {
        this.errorMessage = 'A user with this email already exists';
      } else if (err.status === 429) {
        this.errorMessage = 'Too many attempts. Please wait a minute and try again.';
      } else if (err.status === 400) {
        this.errorMessage = err.error?.message || 'Please check the entered details';
      } else {
        this.errorMessage = 'Registration failed. Please try again later.';
      }
    }
  });
}

/** Account created. If the server asks for OTP, stay on this page and show step 2; otherwise go to login
as before. */
private afterRegister(email: string, mobile: string): void {
  this.otp.status().subscribe({
    next: s => {
      this.loading = false;
      this.otpRequired = !!s.required;
      if (!s.required) { this.router.navigate(['/login']); return; }
      this.email = email;

```

```

        this.maskedMobile = '*****' + String(mobile).slice(-4);
        this.step = 'verify';
        this.sendCode();
    },
    error: () => { this.loading = false; this.router.navigate(['/login']); }
});
}

get canVerify(): boolean {
    return /^[0-9]{6}$/.test(this.code) && !this.verifying;
}

onCodeInput(): void {
    this.code = this.code.replace(/[^0-9]/g, '').slice(0, 6);
}

sendCode(): void {
    if (this.sending || this.secondsLeft > 0) { return; }
    this.sending = true;
    this.otpError = '';
    this.otp.send(this.email).subscribe({
        next: r => {
            this.sending = false;
            this.otpInfo = r.message;
            this.devCode = r.devCode || '';
            this.startCountdown(r.resendInSeconds || 30);
        },
        error: err => {
            this.sending = false;
            this.otpError = [400, 429].includes(err.status) ? (err.error?.message || 'Could not send the
code.')
                : 'The code could not be sent. Please try again in a moment.';
            if (err.status === 429) { this.startCountdown(Number(err.headers?.get('Retry-After')) || 30); }
        }
    });
}

verify(): void {
    if (!this.canVerify) { return; }
    this.verifying = true;
    this.otpError = '';
    this.otp.verify(this.email, this.code).subscribe({
        next: () => { this.verifying = false; clearInterval(this.timer); this.step = 'done'; },
        error: err => {
            this.verifying = false;
            this.code = '';
            this.otpError = [400, 429].includes(err.status) ? (err.error?.message || 'That code did not work.')
                : 'Could not check the code. Please try again.';
        }
    });
}

private startCountdown(seconds: number): void {
    clearInterval(this.timer);
    this.secondsLeft = seconds;
    this.timer = setInterval(() => {
        this.secondsLeft = Math.max(0, this.secondsLeft - 1);
        if (this.secondsLeft === 0) { clearInterval(this.timer); }
    }, 1000);
}
}

```

components/registration/registration.component.html (REPLACE the whole file)

```

<div class="auth">
  <section class="auth-hero">
    <h1>Start learning today.</h1>
    <p>Create a free customer account to browse courses, enroll and review them.</p>
    <ul>

```

```

    <li><mat-icon>person_add</mat-icon>One account, one minute to set up</li>
    <li><mat-icon>lock</mat-icon>Your password is stored securely</li>
    <li><mat-icon>devices</mat-icon>Works on desktop and mobile</li>
  </ul>
</section>

<section class="auth-form">
  <div class="form-card">
    <ol class="steps" id="regSteps" aria-label="Registration steps" *ngIf="otpRequired">
      <li [class.on]="step === 'form'" [class.done]="step !== 'form'"><span>1</span>Your details</li>
      <li [class.on]="step === 'verify'" [class.done]="step === 'done'"><span>2</span>Verify mobile</li>
      <li [class.on]="step === 'done'"><span>3</span>Done</li>
    </ol>

    <ng-container *ngIf="step === 'form'">
      <h2>Registration</h2>
      <p class="lead">Fields marked * are required.</p>
      <p class="form-note" id="otpNote" *ngIf="otpRequired">After you register we send a 6 digit code to
your mobile number. You verify it on the next step, here on this page.</p>
      <form [formGroup]="registerForm" (ngSubmit)="onSubmit()" novalidate>
        <mat-form-field appearance="outline">
          <mat-label>Username</mat-label>
          <input matInput id="username" type="text" formControlName="username" autocomplete="username" />
          <mat-error *ngIf="f['username'].errors?.['required']">Username is required</mat-error>
          <mat-error *ngIf="f['username'].errors?.['minlength']">Username needs at least 3 characters</mat-
error>
          <mat-error *ngIf="f['username'].errors?.['spaces']">Username cannot contain spaces</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="email" />
          <mat-error *ngIf="f['email'].errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="f['email'].errors?.['pattern']">Enter a valid email, for example
name&#64;example.com</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Password</mat-label>
          <input matInput id="password" [type]="showPassword ? 'text' : 'password'"
formControlName="password" autocomplete="new-password" (focus)="passwordFocused = true" />
          <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword"
[attr.aria-label]="showPassword ? 'Hide password' : 'Show password'">
            <mat-icon>{{ showPassword ? 'visibility_off' : 'visibility' }}</mat-icon>
          </button>
          <mat-error *ngIf="f['password'].errors?.['required']">Password is required</mat-error>
          <mat-error *ngIf="f['password'].errors?.['weak']">Password does not meet all the rules
below</mat-error>
        </mat-form-field>
        <ul class="pw-rules" id="passwordRules" aria-label="Password rules" *ngIf="passwordFocused ||
f['password'].value">
          <li *ngFor="let c of passwordChecks" [class.ok]="c.ok">
            <mat-icon aria-hidden="true">{{ c.ok ? 'check_circle' : 'radio_button_unchecked' }}</mat-icon>
            {{ c.label }}
          </li>
        </ul>

        <mat-form-field appearance="outline">
          <mat-label>Confirm Password</mat-label>
          <input matInput id="confirmPassword" [type]="showPassword ? 'text' : 'password'"
formControlName="confirmPassword" autocomplete="new-password" [errorStateMatcher]="confirmMatcher" />
          <mat-error *ngIf="f['confirmPassword'].errors?.['required']">Confirm password is required</mat-
error>
          <mat-error *ngIf="!f['confirmPassword'].errors?.['required'] && registerForm.errors?.
['mismatch']">Password do not match</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Mobile Number</mat-label>
          <input matInput id="mobileNumber" type="text" inputmode="numeric" formControlName="mobileNumber"
maxlength="10" />

```

```

        <mat-error *ngIf="f['mobileNumber'].errors?.['required']">Mobile number is required</mat-error>
        <mat-error *ngIf="f['mobileNumber'].errors?.['pattern']">Enter a 10 digit mobile number starting
with 6, 7, 8 or 9</mat-error>
    </mat-form-field>

    <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
errorMessage }}</div>
    <button mat-flat-button color="primary" type="submit" id="registerButton"
[disabled]="registerForm.invalid || loading">
    {{ loading ? 'Creating account...' : 'Register' }}
</button>
</form>
<ng-container *ngIf="googleEnabled">
    <div class="or"><span>or</span></div>
    <div class="form-error" role="alert" id="googleError" *ngIf="googleError"><mat-
icon>error_outline</mat-icon>{{ googleError }}<a *ngIf="alreadyRegistered" routerLink="/login" class="err-
link" id="loginFromError">Log in</a></div>
    <div #googleButton id="googleButton" class="google-btn"></div>
</ng-container>
<p class="alt">Already have an account? <a routerLink="/login">Login</a></p>
</ng-container>

<!-- step 2: verify the mobile number (only when the server has OTP switched on) -->
<ng-container *ngIf="step === 'verify'">
    <h2 id="stepVerify">Verify your mobile number</h2>
    <p class="lead">Your account <strong>{{ email }}</strong> is created. We sent a 6 digit code by SMS
to <strong>{{ maskedMobile }}</strong>. It works for 5 minutes.</p>

    <p class="form-note" *ngIf="devCode" id="devCodeNote">Development mode: no SMS was sent. Your code
is <strong id="devCode">{{ devCode }}</strong>.</p>
    <div class="form-error" role="alert" *ngIf="otpError"><mat-icon>error_outline</mat-icon>{{ otpError
}}</div>

    <form class="otp-form" (ngSubmit)="verify()" novalidate>
        <label class="field" for="otpCode">Verification code</label>
        <input id="otpCode" name="code" class="code" type="text" inputmode="numeric" autocomplete="one-
time-code"
            maxLength="6" placeholder="-----" [(ngModel)]="code" (ngModelChange)="onCodeInput()"
[disabled]="verifying" />
        <button mat-flat-button color="primary" id="verifyOtp" type="submit" [disabled]="!canVerify">
            {{ verifying ? 'Checking...' : 'Verify' }}
        </button>
    </form>

    <p class="alt">
        Did not get it?
        <button type="button" class="link" id="resendOtp" (click)="sendCode()" [disabled]="sending ||
secondsLeft > 0">
            {{ secondsLeft > 0 ? 'Resend in ' + secondsLeft + 's' : (sending ? 'Sending...' : 'Resend
code') }}
        </button>
    </p>
</ng-container>

<!-- step 3 -->
<ng-container *ngIf="step === 'done'">
    <div class="done-icon"><mat-icon>verified</mat-icon></div>
    <h2 id="regDone">Mobile number verified</h2>
    <p class="lead">Your account is ready. Log in with your email and password.</p>
    <a mat-flat-button color="primary" class="go-login" id="goLogin" routerLink="/login">Log in now</a>
</ng-container>
</div>
</section>
</div>

```

components/registration/registration.component.css (REPLACE the whole file)

```

.auth { display: grid; grid-template-columns: 1.05fr 1fr; min-height: calc(100vh - 64px); }
.auth-hero { display: flex; flex-direction: column; justify-content: center; gap: 22px; padding: 56px

```

```

clamp(24px, 6vw, 88px); color: #e8ecf5; background: linear-gradient(160deg, #0d1730 0%, #1c2c55 70%); }
.auth-hero h1 { color: #fff; font-size: clamp(34px, 4.2vw, 52px); line-height: 1.1; max-width: 14ch; }
.auth-hero p { margin: 0; max-width: 44ch; line-height: 1.65; color: #b9c3d8; }
.auth-hero ul { margin: 0; padding: 0; list-style: none; display: grid; gap: 12px; }
.auth-hero li { display: flex; align-items: center; gap: 12px; color: #dfe5f1; }
.auth-hero li mat-icon { color: #fbb92b; }
.auth-form { display: flex; align-items: center; justify-content: center; padding: 32px 20px; }
.auth-form .form-card { width: 100%; }
.alt { margin: 18px 0 0; text-align: center; color: var(--muted); font-size: 14px; }
@media (max-width: 860px) { .auth { grid-template-columns: 1fr; } .auth-hero { padding: 32px 22px; } .auth-
hero ul { display: none; } .auth-hero h1 { max-width: none; } }

/* live password checklist */
.pw-rules { display: grid; gap: 6px; margin: 2px 0 14px; padding: 12px 14px; list-style: none; border-
radius: 10px; background: var(--surface-2); border: 1px solid var(--line); }
.pw-rules li { display: flex; align-items: center; gap: 8px; color: var(--muted); font-size: 13.5px; }
.pw-rules li mat-icon { width: 18px; height: 18px; font-size: 18px; }
.pw-rules li.ok { color: var(--ok); }

/* step list and the inline OTP step */
.steps { display: flex; gap: 8px; margin: 0 0 18px; padding: 0; list-style: none; flex-wrap: wrap; }
.steps li { display: inline-flex; align-items: center; gap: 8px; padding: 6px 12px 6px 6px; border-radius:
999px; border: 1px solid var(--line); background: var(--surface-2); color: var(--muted); font: 600 13px
Inter, sans-serif; }
.steps li span { display: grid; place-items: center; width: 22px; height: 22px; border-radius: 50%;
background: var(--surface); border: 1px solid var(--line); font-size: 12px; }
.steps li.on { color: #1a1205; background: var(--gold); border-color: var(--gold); }
.steps li.on span { background: rgba(0, 0, 0, .15); border-color: transparent; color: #1a1205; }
.steps li.done { color: var(--ok); border-color: rgba(52, 211, 153, .45); }
.steps li.done span { background: rgba(52, 211, 153, .14); border-color: transparent; color: var(--ok); }
.otp-form { display: flex; flex-direction: column; gap: 12px; margin-top: 16px; }
.field { font: 700 13px var(--display); color: var(--gold-text); }
.code { width: 100%; padding: 14px; text-align: center; font: 700 28px/1 var(--display); letter-spacing:
.5em; text-indent: .5em; color: var(--ink); background: var(--surface-2); border: 1px solid var(--line);
border-radius: 12px; }
.code::placeholder { color: var(--muted); opacity: .5; }
.code:focus { outline: none; border-color: var(--gold); box-shadow: 0 0 0 3px rgba(245, 158, 11, .25); }
.link { border: 0; background: none; padding: 0; color: var(--gold-text); font-weight: 700; cursor:
pointer; text-decoration: underline; text-decoration-offset: 3px; }
.link:disabled { color: var(--muted); text-decoration: none; cursor: default; }
.done-icon { display: grid; place-items: center; width: 56px; height: 56px; margin: 0 0 14px; border-
radius: 16px; background: rgba(52, 211, 153, .14); color: var(--ok); }
.done-icon mat-icon { font-size: 30px; width: 30px; height: 30px; }
.go-login { margin-top: 8px; }

.or { display: flex; align-items: center; gap: 12px; margin: 20px 0 14px; color: var(--muted); font-size:
13px; }
.or::before, .or::after { content: ''; flex: 1; height: 1px; background: var(--line); }
.google-btn { display: flex; justify-content: center; min-height: 44px; }
.err-link { margin-left: auto; font-weight: 700; color: var(--gold-text); }

```

(The separate `/verify-phone` page stays: the login page still sends you there when someone who has not verified tries to log in.)

3d. `components/login/login.component.ts`: in `onSubmit`'s `error`: handler find

```

if (err.status === 401) {
  this.errorMessage = 'Invalid email or password';
} else if (err.status === 429) {

```

and replace with

```

if (err.status === 401) {
  this.errorMessage = 'Invalid email or password';
} else if (err.status === 403 && String(err.error?.message || '').includes('verify your mobile

```

```
number')) {
  this.router.navigate(['/verify-phone'], { queryParams: { email: this.loginForm.value.email } });
} else if (err.status === 429) {
```

(The Google guide also has an `if (err.status === 401)` block inside its own Google handler. Edit only the one in `onSubmit`.)

4. Check it (needs the Day 1 backend with `OTP_REQUIRED=true`, `OTP_DEV_EXPOSE_CODE=true`)

1. Register a new customer: the registration page stays open and switches to **step 2, Verify your mobile number** (the step list at the top moves to step 2), and a note shows the dev code.
2. Type letters: nothing appears. Type 5 digits: Verify stays disabled.
3. Enter a wrong code: "tries left" message. Wait for Resend, press it, enter the right code.
4. After the redirect, log in: works.
5. Register another user, do not verify, close the tab, then log in: you are sent to `/verify-phone`.
6. Switch `OTP_REQUIRED=false`, restart the backend, register: you end on the login page.
7. Narrow the browser to phone width: no sideways scrolling.

5. Troubleshooting

Symptom	Cause
<code>/verify-phone</code> shows the home or error page	<code>VerificationModule</code> is after <code>AppRoutingModule</code>
<code>API.OTP_SEND</code> does not exist	step 3a missed
Page bounces to login at once	OTP is off, or the address has no <code>?email=</code>
403 on login but no redirect	the message text changed; the page looks for "verify your mobile number"

6. Complete code

`services/otp.service.ts` (new file)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';

export interface OtpSendResponse {
  required: boolean;
  message: string;
  resendInSeconds: number;
  devCode?: string | null;
}

export interface OtpVerifyResponse {
  verified: boolean;
  message: string;
}

/** Mobile number verification by OTP (public endpoints: the user cannot log in until the number is
```

```

verified). */
@Injectable({
  providedIn: 'root'
})
export class OtpService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  status(): Observable<{ required: boolean }> {
    return this.http.get<{ required: boolean }>(`${this.apiUrl}${API.OTP_STATUS}`);
  }

  send(email: string): Observable<OtpSendResponse> {
    return this.http.post<OtpSendResponse>(`${this.apiUrl}${API.OTP_SEND}`, { email });
  }

  verify(email: string, code: string): Observable<OtpVerifyResponse> {
    return this.http.post<OtpVerifyResponse>(`${this.apiUrl}${API.OTP_VERIFY}`, { email, code });
  }
}

```

modules/verification/verification.module.ts (new file)

```

import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { SharedModule } from '../shared/shared.module';
import { VerifyPhoneComponent } from '../components/verify-phone/verify-phone.component';

const routes: Routes = [{ path: 'verify-phone', component: VerifyPhoneComponent }];

/**
 * The "Verify your mobile number" page (public). Import it in AppModule BEFORE AppRoutingModule so its
 * route
 * is registered before the "***" fallback.
 */
@NgModule({
  declarations: [VerifyPhoneComponent],
  imports: [SharedModule, RouterModule.forChild(routes)]
})
export class VerificationModule {}

```

components/verify-phone/verify-phone.component.ts (new file)

```

import { Component, OnDestroy, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { OtpService } from '../services/otp.service';

/**
 * "Verify your mobile number" page. It is opened after registration (and when a login is refused because
 * the number
 * is not verified yet) with ?email=... in the address. If the platform does not ask for OTP it sends the
 * user
 * straight to the login page. Otherwise it sends a code once, lets the user type the 6 digits, and offers
 * "Resend" after a short wait.
 */
@Component({
  selector: 'app-verify-phone',
  templateUrl: './verify-phone.component.html',
  styleUrls: ['./verify-phone.component.css']
})
export class VerifyPhoneComponent implements OnInit, OnDestroy {
  email = '';
  code = '';
  checking = true;
  sending = false;
  verifying = false;

```

```

done = false;
error = '';
info = '';
devCode = '';
secondsLeft = 0;
private timer?: any;
private redirect?: any;

constructor(private otp: OtpService, private route: ActivatedRoute, private router: Router) {}

ngOnInit(): void {
  this.email = (this.route.snapshot.queryParamMap.get('email') || '').trim();
  if (!this.email) { this.router.navigate(['/login']); return; }
  this.otp.status().subscribe({
    next: s => {
      if (!s.required) { this.router.navigate(['/login']); return; }
      this.checking = false;
      this.sendCode();
    },
    error: () => { this.checking = false; this.error = 'Could not reach the server. Please try again
later.'; }
  });
}

ngOnDestroy(): void {
  clearInterval(this.timer);
  clearTimeout(this.redirect);
}

get canVerify(): boolean {
  return /^[0-9]{6}$/.test(this.code) && !this.verifying && !this.done;
}

onCodeInput(): void {
  this.code = this.code.replace(/[^0-9]/g, '').slice(0, 6);
}

sendCode(): void {
  if (this.sending || this.secondsLeft > 0) { return; }
  this.sending = true;
  this.error = '';
  this.otp.send(this.email).subscribe({
    next: r => {
      this.sending = false;
      this.info = r.message;
      this.devCode = r.devCode || '';
      this.startCountdown(r.resendInSeconds || 30);
    },
    error: err => {
      this.sending = false;
      this.error = [400, 429].includes(err.status) ? (err.error?.message || 'Could not send the code.')
        : 'The code could not be sent. Please try again in a moment.';
      if (err.status === 429) { this.startCountdown(Number(err.headers?.get('Retry-After')) || 30); }
    }
  });
}

verify(): void {
  if (!this.canVerify) { return; }
  this.verifying = true;
  this.error = '';
  this.otp.verify(this.email, this.code).subscribe({
    next: () => {
      this.verifying = false;
      this.done = true;
      clearInterval(this.timer);
      this.redirect = setTimeout(() => this.router.navigate(['/login']), 2500);
    },
    error: err => {
      this.verifying = false;
      this.code = '';
    }
  });
}

```

```

        this.error = [400, 429].includes(err.status) ? (err.error?.message || 'That code did not work.')
        : 'Could not check the code. Please try again.';
    }
  });
}

private startCountdown(seconds: number): void {
  clearInterval(this.timer);
  this.secondsLeft = seconds;
  this.timer = setInterval(() => {
    this.secondsLeft = Math.max(0, this.secondsLeft - 1);
    if (this.secondsLeft === 0) { clearInterval(this.timer); }
  }, 1000);
}
}

```

components/verify-phone/verify-phone.component.html (new file)

```

<div class="page narrow">
  <section class="form-card verify" *ngIf="!checking || error">
    <div class="icon"><mat-icon>{{ done ? 'verified' : 'sms' }}</mat-icon></div>

    <ng-container *ngIf="!done">
      <h2>Verify your mobile number</h2>
      <p class="muted">We sent a 6 digit code by SMS to the mobile number you gave when you registered with
      <strong>{{ email }}</strong>. The code works for 5 minutes.</p>

      <p class="form-note" *ngIf="devCode" id="devCodeNote">Development mode: no SMS was sent. Your code is
      <strong id="devCode">{{ devCode }}</strong>.</p>
      <div class="form-error" role="alert" *ngIf="error"><mat-icon>error_outline</mat-icon>{{ error }}
    </div>

    <form (ngSubmit)="verify()" novalidate>
      <label class="field" for="otpCode">Verification code</label>
      <input id="otpCode" name="code" class="code" type="text" inputmode="numeric" autocomplete="one-
      time-code"
        maxlength="6" placeholder="-----" [(ngModel)]="code" (ngModelChange)="onCodeInput()"
      [disabled]="verifying" />
      <button mat-flat-button color="primary" class="solid" id="verify0tp" type="submit"
      [disabled]="!canVerify">
        {{ verifying ? 'Checking...' : 'Verify' }}
      </button>
    </form>

    <p class="resend">
      Did not get it?
      <button type="button" class="link" id="resend0tp" (click)="sendCode()" [disabled]="sending ||
      secondsLeft > 0">
        {{ secondsLeft > 0 ? 'Resend in' + secondsLeft + 's' : (sending ? 'Sending...' : 'Resend code')
      }}
    </button>
    </p>
  </ng-container>

  <ng-container *ngIf="done">
    <h2>Mobile number verified</h2>
    <p class="muted" id="verifiedNote">Thank you. Taking you to the login page...</p>
    <a mat-flat-button color="primary" class="solid" routerLink="/login">Log in now</a>
  </ng-container>
</section>
</div>

```

components/verify-phone/verify-phone.component.css (new file)

```

/* Verify phone (Midnight) */
.verify { max-width: 480px; margin: 24px auto 0; text-align: center; }
.icon { display: grid; place-items: center; width: 56px; height: 56px; margin: 0 auto 14px; border-radius:

```

```
16px; background: var(--gold-soft); color: var(--gold-text); }
.icon mat-icon { font-size: 30px; width: 30px; height: 30px; }
h2 { font-size: 24px; margin-bottom: 8px; }
.muted { color: var(--muted); line-height: 1.6; }
.muted strong { color: var(--ink-2); }
form { display: flex; flex-direction: column; gap: 12px; margin-top: 18px; text-align: left; }
.field { font: 700 13px var(--display); color: var(--gold-text); }
.code { width: 100%; padding: 14px; text-align: center; font: 700 28px/1 var(--display); letter-spacing:
.5em; text-indent: .5em; color: var(--ink); background: var(--surface-2); border: 1px solid var(--line);
border-radius: 12px; }
.code::placeholder { color: var(--muted); opacity: .5; }
.code:focus { outline: none; border-color: var(--gold); box-shadow: 0 0 0 3px rgba(245, 158, 11, .25); }
.resend { margin: 18px 0 0; color: var(--muted); font-size: 14px; }
.link { border: 0; background: none; padding: 0; color: var(--gold-text); font-weight: 700; cursor:
pointer; text-decoration: underline; text-underline-offset: 3px; }
.link:disabled { color: var(--muted); text-decoration: none; cursor: default; }
.form-note, .form-error { text-align: left; margin-top: 14px; }
```